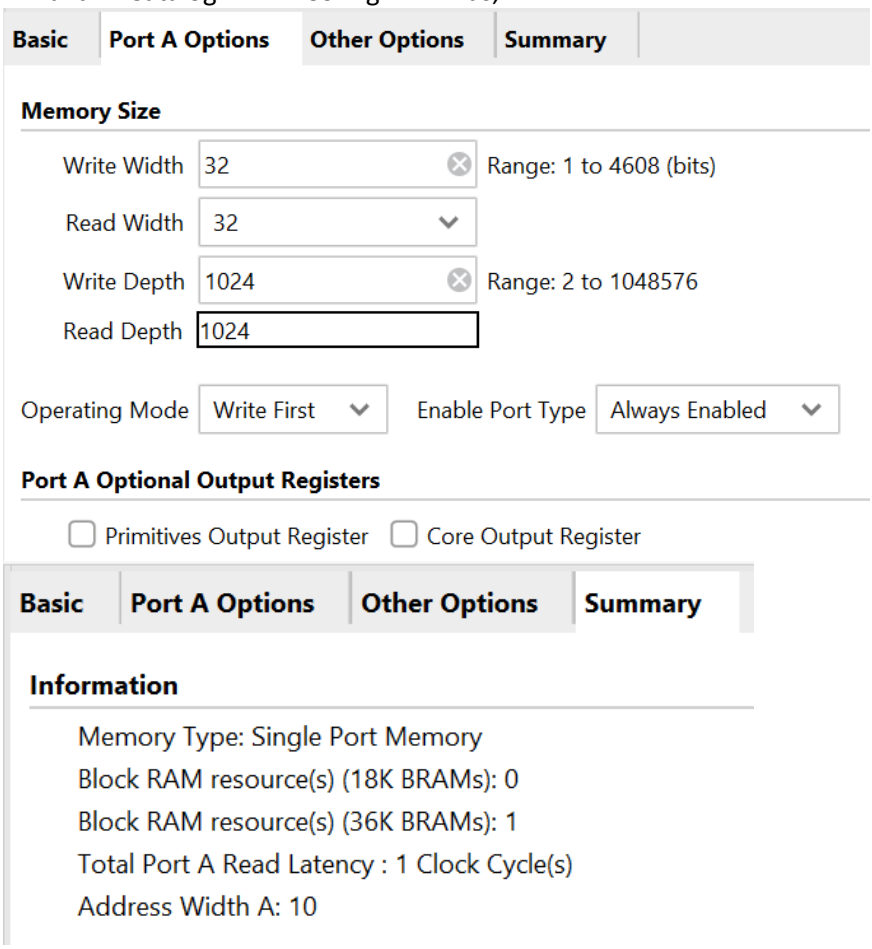
Stack Process 101 – Develop new machine instructions

1. Build user logic stack processor 101

2. Build IP Catalog BRAM Config BRAM as,



3. Add the bridge code

```
----------------------------------------
-- Module Name:    bus_ip_mem_bridge - Behavioral
-- Additional Comments: A bridge between bus or user ip to block ram memory
-- input signals are data and addresses from bus an ip to be muxed to mem
-- bus2mem_we,ip2mem_we are write enable from bus and mem
-- bus2mem_en is from bus controls the access to mem
-- output signals are to mem
-- busy signal is to ip indicating that bus is accessing memory
----------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
library UNISIM;
use UNISIM.VComponents.all;
```

```
entity bus_ip_mem_bridge is
generic (A: natural := 10);
port(ip2mem_data_in,bus2mem_data_in : in  std_logic_vector(31 downto 0);
          ip2mem_addr,bus2mem_addr  : in  std_logic_vector(A-1 downto 0);
               bus2mem_we,ip2mem_we : in  std_logic_vector(0  downto 0);
                         bus2mem_en : in  std_logic;
                              addra : out std_logic_vector(A-1 downto 0);
                               dina : out std_logic_vector(31 downto 0);
                                wea : out std_logic_vector(0 downto 0);
                                  busy : out std_logic);
end bus_ip_mem_bridge;

architecture Behavioral of bus_ip_mem_bridge is
begin
process(bus2mem_addr,bus2mem_data_in,
ip2mem_addr,ip2mem_data_in,bus2mem_we,ip2mem_we,
bus2mem_en)
begin
if bus2mem_en = '1' then
    wea <= bus2mem_we;
    addra <= bus2mem_addr;
    dina <= bus2mem_data_in;
     busy <= '1';
else
    wea <= ip2mem_we;
    addra <= ip2mem_addr;
    dina <= ip2mem_data_in;
     busy <= '0';
end if;
end process;
end Behavioral;
```

Code below is for machine instruction copy – scp.

```
------------------------------
-- Module Name: sp101
-- Comments: Stack Processor fetch-exe constant load store add
------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.NUMERIC_STD.ALL;
library UNISIM;
use UNISIM.VComponents.all;

entity user_logic is
generic (A : natural := 10);
port (
run,reset,bus2mem_en,bus2mem_we,ck : in  std_logic;
```

```
                      bus2mem_addr : in  std_logic_vector(A-1 downto 0);
                    bus2mem_data_in : in  std_logic_vector(31 downto 0);
                    sp2bus_data_out : out std_logic_vector(31 downto 0);
                               done : out std_logic);
end user_logic;

architecture Behavioral of user_logic is

-- bram signals
-- wea is bram port we is processor signal
signal wea, we : STD_LOGIC_VECTOR(0 DOWNTO 0);
signal addra   : STD_LOGIC_VECTOR(A-1 DOWNTO 0);
signal dina    : STD_LOGIC_VECTOR(31 DOWNTO 0);
signal douta   : STD_LOGIC_VECTOR(31 DOWNTO 0);
--cast bus2mem_we:in std_logic to wea:std_logic_vector
signal temp_we   : STD_LOGIC_VECTOR(0 DOWNTO 0);

-------------------------
-- processor registers
-------------------------

-- pointers
signal sp,pc,mem_addr : std_logic_vector(A-1 downto 0);

-- data registers
signal mem_data_in,mem_data_out,ir : std_logic_vector(31 downto 0);
signal temp1,temp2 : std_logic_vector(31 downto 0);

-- flags
signal busy, done_FF: std_logic;

------------------
-- machine state
------------------
type state is (idle,fetch,fetch2,
fetch3, --implementation comment out fetch3,
fetch4,fetch5,exe,chill);
signal n_s: state;

-----------------------------------
-- Instruction Definitions
-- Rightmost hex is the step in an instruction
-- higher hex is the code for an instruction,
-- e.g., the steps in SC (constant) instruction 0x01,.., 0x05
-- the steps in sl (load from memory) 0x11 to 0x19
-----------------------------------

constant HALT : std_logic_vector(31 downto 0) := (x"000000FF");

constant SC   : std_logic_vector(31 downto 0) := (x"00000001");
```

```
constant SC2  : std_logic_vector(31 downto 0) := (x"00000002");
-- implementation comment out sc3
constant SC3  : std_logic_vector(31 downto 0) := (x"00000003");
constant SC4  : std_logic_vector(31 downto 0) := (x"00000004");
constant SC5  : std_logic_vector(31 downto 0) := (x"00000005");

 constant sl   : std_logic_vector(31 downto 0) := (x"00000011");
 constant sl2  : std_logic_vector(31 downto 0) := (x"00000012");
-- implementation comment out sl3
 constant sl3  : std_logic_vector(31 downto 0) := (x"00000013");
 constant sl4  : std_logic_vector(31 downto 0) := (x"00000014");
 constant sl5  : std_logic_vector(31 downto 0) := (x"00000015");
 constant sl6  : std_logic_vector(31 downto 0) := (x"00000016");
-- implementation comment out sl7
 constant sl7  : std_logic_vector(31 downto 0) := (x"00000017");
 constant sl8  : std_logic_vector(31 downto 0) := (x"00000018");
 constant sl9  : std_logic_vector(31 downto 0) := (x"00000019");

 constant ss   : std_logic_vector(31 downto 0) := (x"00000021");
 constant ss2  : std_logic_vector(31 downto 0) := (x"00000022");
-- implementation comment out ss3
 constant ss3  : std_logic_vector(31 downto 0) := (x"00000023");
 constant ss4  : std_logic_vector(31 downto 0) := (x"00000024");
 constant ss5  : std_logic_vector(31 downto 0) := (x"00000025");
 constant ss6  : std_logic_vector(31 downto 0) := (x"00000026");

 constant sadd : std_logic_vector(31 downto 0) := (x"00000031");
 constant sadd2: std_logic_vector(31 downto 0) := (x"00000032");
 -- implementation comment out sadd3
 constant sadd3: std_logic_vector(31 downto 0) := (x"00000033");
 constant sadd4: std_logic_vector(31 downto 0) := (x"00000034");
 constant sadd5: std_logic_vector(31 downto 0) := (x"00000035");
 constant sadd6: std_logic_vector(31 downto 0) := (x"00000036");

 constant scp  : std_logic_vector(31 downto 0) := (x"00000101");
 constant scp2 : std_logic_vector(31 downto 0) := (x"00000102");
 -- implementation comment out scp3
 constant scp3 : std_logic_vector(31 downto 0) := (x"00000103");
 constant scp4 : std_logic_vector(31 downto 0) := (x"00000104");
 constant scp5 : std_logic_vector(31 downto 0) := (x"00000105");
 constant scp6 : std_logic_vector(31 downto 0) := (x"00000106");
-- implementation comment out scp7
 constant scp7 : std_logic_vector(31 downto 0) := (x"00000107");
 constant scp8 : std_logic_vector(31 downto 0) := (x"00000108");
 constant scp9 : std_logic_vector(31 downto 0) := (x"00000109");
------------------
-- components
------------------
COMPONENT blk_mem_gen_0
  PORT (
```

```
    clka : IN STD_LOGIC;
    wea : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
    addra : IN STD_LOGIC_VECTOR(9 DOWNTO 0);
    dina : IN STD_LOGIC_VECTOR(31 DOWNTO 0);
    douta : OUT STD_LOGIC_VECTOR(31 DOWNTO 0)
  );
END COMPONENT;
-- MEM bridge multiplexes BUS or processor signals to bram.
-- It also flags busy signal.
component  bus_ip_mem_bridge
generic(A: natural := 10); -- A-bit Address
port(
ip2mem_data_in,bus2mem_data_in : in  std_logic_vector(31 downto 0);
     ip2mem_addr,bus2mem_addr  : in  std_logic_vector(A-1 downto 0);
          bus2mem_we,ip2mem_we : in  std_logic_vector(0  downto 0);
                    bus2mem_en : in  std_logic;
                           addra : out std_logic_vector(A-1 downto 0);
                                                                dina : out
std_logic_vector(31 downto 0);
                           wea : out std_logic_vector(0 downto 0);
                          busy : out std_logic);
end component;

begin

----------------------------
-- components instantiation
----------------------------

temp_we(0) <= bus2mem_we; -- wire bus2mem_we to an std_logic_vector

-- MEM bridge multiplexes BUS or processor signals to bram.
bridge: bus_ip_mem_bridge -- It also flags busy signal to processor.
generic map(A)
port map(bus2mem_addr => bus2mem_addr,
      bus2mem_data_in => bus2mem_data_in,
          ip2mem_addr => mem_addr,
       ip2mem_data_in => mem_data_in,
           bus2mem_we => temp_we,
            ip2mem_we => we,
           bus2mem_en => bus2mem_en,
                addra => addra,
                 dina => dina,
                  wea => wea,
                 busy => busy);

-- main memory
mm : blk_mem_gen_0 PORT MAP (clka => ck,
                         wea => wea,
                       addra => addra,
```

```
                        dina => dina,
                       douta => douta);

-- memory data out register
process(ck)
begin
if ck='1' and ck'event then
  if reset = '1' then mem_data_out <= (others => '0');
  else mem_data_out <= douta;
  end if;
end if;
end process;

 -- wire to output ports sp2bus_data_out
sp2bus_data_out <= mem_data_out; done <= done_FF;

-------------------
-- Stack Processor
-------------------

process(ck)

begin
if ck='1' and ck'event then
  if reset='1' then n_s <= idle; else
                  --           Machine State Diagram
  case n_s is     --              run               halt
    when chill =>--reset~~>(idle)-->(fetch)-->(exe)-->(chill)
                      null; --                    |        |
              --                     |        v
              --                      <----(case ir)
    when idle =>
                              pc <= (others => '0');
                sp <= (7 => '1', others => '0'); -- stack base 128
            ir <= (others => '0');
         temp1 <= (others => '0'); temp2 <= (others => '0');
      mem_addr <= (others => '0');
  mem_data_in <= (others => '0');
           we <= "0"; done_FF <= '0';

          -- poll on run and not busy
              if run='1' and busy='0' then n_s <= fetch; end if;

     when fetch => -- "init" means to initiate an action
             mem_addr <= pc; pc <= pc+1;--init load pc to mem_addr
                                        we <= "0"; -- enable read next
state
                                        n_s <= fetch2;
        when fetch2 => -- mem_addr valid, pc advanced
                                        we <= "0";    -- read
```

```
               n_s <= fetch3; -- one additional latency when simulate
--               n_s <= fetch4; -- HW ip skip fetch3
                                                -----
                                                ----- mem_addr
      when fetch3 => -- mem read latency=1        |   register
             we <= "0"; -- read                --------
            n_s <= fetch4;--                  |  BRAM  |
        when fetch4 => -- douta valid          --------
               we <= "0"; -- read                  | dout
              n_s <= fetch5; --                   -----
        when fetch5 => -- mem_data_out valid  ----- mem_data_out
               we <= "0"; -- read                  |   register
                         ir <= mem_data_out;-- init ir load
              n_s <= exe;

     when exe =>   -- ir loaded
        case ir is -- Machine Instructions

              when halt => -- signal done output and go to chill
          done_FF <= '1'; n_s <= chill;

         -- Stack Constant, init load constant pointed to by pc
                                  when sc =>
         mem_addr <= pc;  --pc points at constant
               pc <= pc+1;--advance to next instruction
               we <= "0"; --enable read next state
               ir <= sc2;
           when sc2 => -- mem_addr valid
                                           we <= "0"; -- read
                                           ir <= sc3; -- one additional
latency when simulate
              -- ir <= sc4; -- HW ip skip sc3
           when sc3 => -- douta not valid latency 1
                                           we <= "0"; -- read
                                           ir <= sc4;
           when sc4 => -- douta valid
                 we <= "0"; -- read
                 ir <= sc5;
           when sc5 => -- mem_data_out valid
           mem_addr <= sp; sp <= sp+1;
         mem_data_in <= mem_data_out;
                 we <= "1"; -- write enable next state
                n_s <= fetch;

        --Load data from memory:pop address,read and stack data
                                  when sl =>
          mem_addr <= sp-1; sp <= sp-1;--init pop data address
                we <= "0"; -- enable read next state
                                          ir <= sl2;
          when sl2 => -- mem_addr updated
```

```
                                          we <= "0"; -- read
                                          ir <= sl3; -- one additional
latency when simulate
--              ir <= sl4; -- HW ip skip sl3
          when sl3 => -- douta not valid latency 1
                ir <= sl4;
          when sl4 => -- douta valid
                ir <= sl5;
          when sl5 => -- mem_data_out valid
             mem_addr <= mem_data_out(A-1 downto 0);--data Address
                ir <= sl6;
          when sl6 => -- mem_addr updated
                ir <= sl7; -- one additional latency when simulate
--              ir <= sl8; -- HW ip skip sl7
          when sl7 => -- douta not valid latency 1
                ir <= sl8;
          when sl8 => -- douta valid
                ir <= sl9;
          when sl9 => -- mem_data_out valid
          mem_addr <= sp; sp <= sp+1;
       mem_data_in <= mem_data_out;--data read
               we <= "1"; -- write enable in next state
               n_s <= fetch;

       --Store data to memory:pop data,address,write to memory
       when ss =>
          mem_addr <= sp-1; sp <= sp-1;--init1 pop data
                ir <= ss2;
          when ss2 =>  -- mem_addr updated1
          mem_addr <= sp-1; sp <= sp-1;--init2 pop address
                ir <= ss3; -- one additional latency when simulate
--              ir <= ss4; -- HW ip skip ss3
          when ss3 =>      --douta1 not valid latency 1,
                ir <= ss4;
          when ss4 =>      -- douta valid1,
                ir <= ss5;
          when ss5 =>  --douta valid2, mem_data_out valid1
             temp1 <= mem_data_out;--temp <= data
                ir <= ss6;
```

```
          when ss6 =>  -- mem_data_out valid2
          mem_addr <= mem_data_out(A-1 downto 0);--init write
       mem_data_in <= temp1; --data in temp1
                we <= "1"; -- write enable in next state
               n_s <= fetch;

       -- Add - pop operands add and push
       when sadd =>
          mem_addr <= sp-1;sp <= sp-1;--init1 pop operand1
                we <= "0"; -- read
                ir <= sadd2;
          when sadd2 => -- mem_addr updated1
               mem_addr <= sp-1; sp <= sp-1;--init2 pop operand2
                 we <= "0"; -- read
                 ir <= sadd3; -- one additional latency when simulate
              ir <= sadd4; -- HW ip skip sadd3
          when sadd3 =>--douta1 not valid latency 1,mem_addr updated2
              we <= "0"; -- read
              ir <= sadd4;
          when sadd4 =>  -- douta valid1, douta2 not valid latency 1,
              we <= "0"; -- read
              ir <= sadd5;
          when sadd5 =>  -- douta valid2, mem_data_out valid1
              we <= "0"; -- read
           temp1 <= mem_data_out;--temp1 <= operand1
              ir <= sadd6;
          when sadd6 => -- mem_data_out valid2
        mem_addr <= sp; sp <= sp+1; -- init push
     mem_data_in <= temp1+mem_data_out;--operand1+operand2
                we <= "1";  -- write enable in next state
              n_s <= fetch;

-- New instruction scp top of stack source addr then dest addr
 when scp =>
          mem_addr <= sp-1; sp <= sp-1;--init pop source address
                we <= "0"; -- enable read next state
                ir <= scp2;
          when scp2 => -- mem_addr updated
          mem_addr <= sp-1; sp <= sp-1;--init pop dest address
                we <= "0"; -- read
               ir <= scp3; -- one additional latency when simulate
--              ir <= scp4; -- HW ip skip scp3
          when scp3 => -- douta not valid latency 1
                we <= "0"; -- read
                ir <= scp4;
          when scp4 => -- douta valid
                we <= "0"; -- read
                ir <= scp5;

when scp5 => -- mem_data_out valid source addr, init read from source addr
```

```
            mem_addr <= mem_data_out(A-1 downto 0);--source Address
                we <= "0"; -- read
                ir <= scp6;
           when scp6 => -- mem_addr updated, mem_data_out valid dest addr
              we <= "0"; -- read mem_addr update
 temp1 <= "00000000000000000000"&mem_data_out(A-1 downto 0);-- dest Address
              ir <= scp7; -- one additional latency when simulate
--              ir <= scp8; -- HW ip skip scp7
           when scp7 => -- douta not valid latency 1,
                we <= "0"; -- read
                ir <= scp8;
           when scp8 => -- douta valid data from source addr
                we <= "0"; -- read
                ir <= scp9;
           when scp9 => -- mem_data_out valid
           mem_addr <= temp1(A-1 downto 0); -- temp1 is dest addr
        mem_data_in <= mem_data_out;--source data
                we <= "1"; -- write enable in next state
               n_s <= fetch;
        when others =>null;
       end case; -- instructions
      end case;  -- fetch-execute
  end if;        -- reset fence
end if;          -- clock fence
end process;
end Behavioral;
```

Simulation Script

```
#scp code is 0x101 = 257
#Addr 100 has 78 copy to Addr 200
# write bus2mem_data_in = 78 to bus2mem_addr = 100
# program dest=200 source=100
# sc 200 sc 100 scp   halt
# 1  200 1  100 0x101  0xff


restart
add_force {/user_logic/ck} -radix hex {1 0ns} {0 50ps} -repeat_every 100ps

#reset
add_force {/user_logic/run} -radix hex {0 0ns}
add_force {/user_logic/reset} -radix hex {1 0ns}
add_force {/user_logic/bus2mem_en} -radix hex {1 0ns}
add_force {/user_logic/bus2mem_we} -radix hex {1 0ns}
add_force {/user_logic/bus2mem_addr} -radix unsigned {0 0ns}
add_force {/user_logic/bus2mem_data_in} -radix unsigned {0 0ns}
run 100 ps

#lower reset
```

```
add_force {/user_logic/reset} -radix hex {0 0ns}
# write datum to BRAM addr 100
add_force {/user_logic/bus2mem_addr} -radix unsigned {100 0ns}
add_force {/user_logic/bus2mem_data_in} -radix unsigned {78 0ns}
run 100 ps

#load program sc 200 sc 100 scp halt (sc=1, scp=257, halt=255)
add_force {/user_logic/bus2mem_addr} -radix unsigned {0 0ns}
add_force {/user_logic/bus2mem_data_in} -radix unsigned {1 0ns}
run 100 ps
add_force {/user_logic/bus2mem_addr} -radix unsigned {1 0ns}
add_force {/user_logic/bus2mem_data_in} -radix unsigned {200 0ns}
run 100 ps
add_force {/user_logic/bus2mem_addr} -radix unsigned {2 0ns}
add_force {/user_logic/bus2mem_data_in} -radix unsigned {1 0ns}
run 100 ps
add_force {/user_logic/bus2mem_addr} -radix unsigned {3 0ns}
add_force {/user_logic/bus2mem_data_in} -radix unsigned {100 0ns}
run 100 ps
add_force {/user_logic/bus2mem_addr} -radix unsigned {4 0ns}
add_force {/user_logic/bus2mem_data_in} -radix unsigned {257 0ns}
run 100 ps
add_force {/user_logic/bus2mem_addr} -radix unsigned {5 0ns}
add_force {/user_logic/bus2mem_data_in} -radix unsigned {255 0ns}
run 100 ps

# run = 1, bus2mem_en = 0, bus2mem_we = 0

add_force {/user_logic/run} -radix hex {1 0ns}
add_force {/user_logic/bus2mem_en} -radix hex {0 0ns}
add_force {/user_logic/bus2mem_we} -radix hex {0 0ns}
run 100 ps

### run until n_s is chill

# read from addr 200
add_force {/user_logic/run} -radix hex {0 0ns}
add_force {/user_logic/bus2mem_en} -radix hex {1 0ns}
add_force {/user_logic/bus2mem_we} -radix hex {0 0ns}
add_force {/user_logic/bus2mem_addr} -radix unsigned {200 0ns}
run 100 ps
```

Test App

```
#include <stdio.h>
#include "platform.h"
#include "xparameters.h"
#include "sp101.h"
#include "xil_io.h"
#include "xil_types.h"
```

```
#define  BASEADDR        XPAR_SP101_0_S_AXI_BASEADDR
#define  sc                              0x01
#define  sl                              0x11
#define  ss                              0x21
#define  sadd                      0x31
#define  scp    0x101
#define  shalt                     0xff
#define  prog_length6 // program1 5, program2 9, program3 15,program4 6
//----------------------------------------------------------------------
// Registers/Ports Mapping
// ======================
// run=>slv_reg0(0),reset=>slv_reg0(1),
// bus2mem_en=>slv_reg0(2),bus2mem_we=>slv_reg0(3),
// bus2mem_addr=>slv_reg1(9 downto 0),
// bus2mem_data_in=>slv_reg2,
// sp2bus_data_out=>slv_reg3,
// done=>slv_reg4(0)
//----------------------------------------------------------------------
/* Machine Instructions
 * -------------------
 constant sc   (x"00000001");
 constant sl   (x"00000011");
 constant ss   (x"00000021");
 constant sadd (x"00000031");
 */
int main()
{

    init_platform();
    //----------------------------------------------------------------
    // slv_reg0 write = reset + bus2mem_en + bus2mem_we = 2+4+8 = 0xE
    //----------------------------------------------------------------
    SP101_mWriteReg(BASEADDR, 0, 0x0000000E);
    //----------------------------------------------------------------
    // Lower Reset: slv_reg0 write = bus2mem_en + bus2mem_we = 4+8 = 0xC
    //----------------------------------------------------------------
    SP101_mWriteReg(BASEADDR, 0, 0x0000000C);
    //---------------------------------------
    // loading user code
    //---------------------------------------
    // # Program 1 stack 100 and 200
    // #  0    1    2    3    4
    // # {sc,  100, sc,  200, shalt}
    //---------------------------------------
    // # Program 2 stores 100 at addr 20 and load content of addr 20 on stack
    // #  0   1   2   3    4   5   6   7   8
    // # {sc, 20, sc, 100, ss, sc, 20, sl, shalt}
    //---------------------------------------
    // Program 3 (i is location 40)
    // # int i = 5;     i++;
```

```
    // #  0  1  2  3 4  5  6  7  8  9  10 11 12   13 14
    // # {sc,40,sc,5,ss,sc,40,sc,40,sl,sc, 1, sadd,ss,shalt}
    // -------------------------------------
    // Program 4
    // #  0  1   2  3    4    5
    // # {sc,100,sc,200, scp, shalt}

//    {sc,  100, sc,  200, shalt};
//    {sc, 20, sc, 300, ss, sc, 20, sl, shalt};
//    {sc,40,sc,5,ss,sc,40,sc,40,sl,sc,1,sadd,ss,shalt};
//    {sc,100,sc,200, scp, shalt};

//    Program 4 write 78 to addr 100
                    SP101_mWriteReg(BASEADDR, 4, 100);
                     SP101_mWriteReg(BASEADDR, 8, 78);
        SP101_mWriteReg(BASEADDR, 4, 100);
        xil_printf("content of %d : %d\n\r", 100 , (unsigned int)
        SP101_mReadReg(BASEADDR,12));

unsigned int prog[prog_length] = {sc,200,sc,100, scp, shalt};
    for (int i = 0; i < prog_length; i++) {
                      SP101_mWriteReg(BASEADDR, 4, i);
                  SP101_mWriteReg(BASEADDR, 8, prog[i]);
    }

    //--------------- END USER CODE ---------------------------

    //----------------------- RUN -----------------------------

    // Lower bus2mem_en, bus2mem_we and run = slv_reg0(0) = 0x1
    SP101_mWriteReg(BASEADDR, 0, 0x1);

    // wait on done flag slv_reg4(0)
    while ((unsigned int) (SP101_mReadReg(BASEADDR,16)&0x1)==0);

    //-------------------- READ RESULTS -----------------------------

    // enable bus2mem_en: slv_reg(2) = 0x4 run and bus2mem_we = 0
    sp101_mWriteReg(BASEADDR, 0, 0x4);

    unsigned int addr = 200; // program1 128, program 128, program3 40
program4 200
    sp101_mWriteReg(BASEADDR, 4, addr);
    xil_printf("content of %d : %d\n\r", addr, (unsigned int)
    SP101_mReadReg(BASEADDR,12));

    cleanup_platform();
    return 0;
}
```